纸牌游戏程序设计

作者：展聪

目录

1. 引言 2
2. 整体架构概述 2
3. 目录结构与核心类说明 3
4. 可扩展性—添加一个新卡牌类型 4
4.1 修改枚举定义（utils/GameEnum.h） 4
4.2 调整匹配逻辑（PlayFieldController.cpp） 5
4.3 调整视图显示（CardView.cpp） 5
5. 可扩展性—添加新类型的回退功能（多张牌同时回退） 6
5.1 扩展撤销记录结构（UndoRecord.h） 6
5.2 在控制器中生成多牌回退的撤销记录 7
5.3 修改视图层以支持多牌动画 8
5.4 在撤销管理器处理多牌回退 8
6. 结论 9

1. 引言

本文档旨在阐述本纸牌游戏项目的程序设计架构、核心模块职责以及未来的扩展性。项目采用 MVC（Model-View-Controller） 架构模式，将数据、视图和业务逻辑分离，以提高代码的可维护性和可扩展性。本文将重点说明如何在此基础上添加新的卡牌类型以及新的回退操作类型。

2. 整体架构概述

项目遵循清晰的分层设计，主要包含以下层次：

Model（数据模型层）：负责存储游戏运行时的所有数据状态，如卡牌属性、牌堆结构、游戏进度等。模型层不包含业务逻辑，仅提供数据存取接口。

View（视图层）：负责卡牌及游戏界面的显示，接收用户输入事件（如点击），并通过回调通知控制器。视图层不处理业务逻辑，仅做 UI 展示。

Controller（控制器层）：作为模型和视图的桥梁，处理用户操作对应的业务逻辑，更新模型状态，并调用视图的动画接口进行反馈。

Manager（管理器层）：作为控制器的辅助组件，提供特定功能的封装（如撤销管理器），可持有模型数据进行加工。

Service（服务层）：提供无状态的业务功能，如根据配置生成游戏模型，不持有数据，可静态调用。

Config（配置层）：负责加载静态配置数据（如关卡配置、资源路径），为游戏初始化提供输入。

这种分层确保了各模块职责单一，修改或扩展某个功能时不会影响其他部分。

3. 目录结构与核心类说明

整体项目 Classes 文档下的目录结构如下所示：

```
Classes/
├──── configs/                                  //静态配置相关
│    ├──── models/                               // 配置数据结构
│    │    └──── LevelConfig.h                   //关卡配置结构
│    └──── loaders/                              //配置加载器
│         └──── LevelConfigLoader.h/cpp          //配置加载器
├──── models/                                   //运行时数据模型
│    ├──── CardModel.h/cpp                       //单张卡牌数据
│    ├──── GameModel.h/cpp                       //游戏全局数据
│    └──── UndoRecord.h                          //撤销记录结构
├──── views/                                    //视图组件
│    ├──── CardView.h/cpp                        //卡牌视图
│    └──── GameView.h/cpp                        //游戏主视图
├──── controllers/                              //控制器
│    ├──── GameController.h/cpp                  //主控制器，协调全局
│    ├──── PlayFieldController.h/cpp             //主牌区控制器
│    ├──── TrayController.h/cpp                  //手牌区控制器
│    └──── StockController.h/cpp                 //备用牌堆控制器
├──── managers/                                 //管理器
│    └──── UndoManager.h/cpp                     //撤销管理器
├──── services/                                 //服务层
│    └──── GameModelFromLevelGenerator.h/cpp     //根据配置生成游戏模型
└──── utils/                                    //工具类
     └──── GameEnum.h                            //公共枚举定义
```

核心类职责：

CardModel：卡牌数据的原子单元，包含唯一 ID、花色、点数、坐标、所在区域及是否翻开。

GameModel：聚合所有卡牌，提供按 ID 查找、获取手牌区顶部牌等方法，并包含布局计算函数（layoutStockCards, layoutPlayfieldCards）。

CardView：组合显示卡牌背景、大数字、小数字和花色，处理触摸事件并通过回调向上传递。

GameView：持有所有 CardView，提供播放各种动画的接口（如交换、移动到顶、抽牌等）。

UndoRecord：记录一次操作前后的状态（涉及两张牌的 ID 和位置），用于撤销。

UndoManager：以栈形式存储撤销记录，提供压栈、弹栈、判断是否可撤销等功能。

各区域控制器：分别处理对应区域卡牌的点击事件，执行匹配或抽牌逻辑，生成撤销记录并通知视图播放动画。

GameController：整合所有子控制器，加载配置，生成模型，创建视图，设置点击回调，并处理全局撤销按钮事件。

4. 可扩展性—添加一个新卡牌类型

假设我们要增加一种“万能牌”（CFT_Joker），它可以替代任何点数进行匹配。

扩展步骤如下：

4.1 修改枚举定义（utils/GameEnum.h）

打开并修改 GameEnum.h 文件，在 CardFaceType 枚举中添加 CFT_JOKER 值：

```
enum class CardFaceType {
    CFT_NONE = -1,
    CFT_ACE,
    CFT_TWO,
    CFT_THREE,
    CFT_FOUR,
    CFT_FIVE,
    CFT_SIX,
    CFT_SEVEN,
    CFT_EIGHT,
    CFT_NINE,
```

```
    CFT_TEN,
    CFT_JACK,
    CFT_QUEEN,
    CFT_KING,
    CFT_JOKER,        // 此为新增“万能牌”
    CFT_NUM_CARD_FACE_TYPES
};
```

4.2 调整匹配逻辑（PlayFieldController.cpp）

打开并修改 PlayFieldController.cpp 文件，找到 onCardClicked，判断如果桌面牌或手牌区顶部牌为 CFT_JOKER，则允许匹配。

如下所示：

```
bool PlayFieldController::onCardClicked(int cardId) {
    //获取 clickedCard 和 topTray
    if (clickedCard->getFace() == CardFaceType::CFT_JOKER || topTray->getFace() == CardFaceType::CFT_JOKER) {
        // 万能牌逻辑：允许匹配，万能牌可以与任何牌匹配
    } else {
        int face1 = static_cast<int>(clickedCard->getFace());
        int face2 = static_cast<int>(topTray->getFace());
        if (std::abs(face1 - face2) != 1) return false;
    }
    //
}
```

4.3 调整视图显示（CardView.cpp）

打开并修改 CardView.cpp 文件，找到 createFaceSprites，为 CFT_JOKER 加载特殊资源。可以在 faceStr 映射中添加：

```
case CardFaceType::CFT_JOKER: faceStr = "JOKER"; break;
```

然后确保资源目录下有对应图片（big_black_JOKER.png 和 small_black_JOKER.png）。

总结：通过以上三步，无需修改框架，仅在枚举、匹配逻辑和视图加载处添加少量代码，即可支持新卡牌。这体现了系统的高内聚低耦合特性。

5. 可扩展性—添加新类型的回退功能（多张牌同时回退）

本项目中所制作的纸牌游戏从本质上讲很类似现在比较火的消消乐休闲小游戏，那我们发挥一下创造力，是否可以将纸牌游戏与消消乐中的奖励回合所联系，当桌牌区存在多张牌能与手牌区的牌相消，那么直接进入到纸牌奖励回合，多张牌同时相抵消，以此来获取大量分数，给游玩者提供兴奋体验。

在具体实现“同时移动多张牌”的操作中，需要同时记录多张牌的移动，撤销时需同时恢复所有牌的位置。本节仅对同时移动多张牌的回退功能展开详细讲解，如下：

5.1 扩展撤销记录结构（UndoRecord.h）

当前 UndoRecord 只记录两张牌的 ID 和位置。为支持多张牌，可以增加一个数组字段，并区分操作类型：

```
enum class UndoType {
    TRAY_SWAP,
    PLAYFIELD_TO_TRAY,
    STOCK_TO_TRAY,
    MULTI_MOVE        // 新增多牌移动类型
};
```

```
struct UndoRecord {
    UndoType type;
    std::vector<int> cardIds;          // 参与操作的牌 ID 列表
    std::vector<cocos2d::Vec2> oldPos;  // 移动前位置列表
    std::vector<cocos2d::Vec2> newPos;  // 移动后位置列表
};
```

5.2 在控制器中生成多牌回退的撤销记录

在 PlayFieldController 中新增一个方法 onMultiCardsClicked，处理多牌选择逻辑，并生成 MULTI_MOVE 记录：

```
void PlayFieldController::onMultiCardsClicked(const
std::vector<int>& cardIds) {
    //记录旧位置
    std::vector<Vec2> oldPos, newPos;
    for (int id : cardIds) {
        CardModel* card = _model->getCardById(id);
        oldPos.push_back(card->getPosition());
        // 计算新位置
        newPos.push_back(targetPos);
    }
    //创建撤销记录
    UndoRecord record;
    record.type = UndoType::MULTI_MOVE;
    record.cardIds = cardIds;
    record.oldPos = oldPos;
    record.newPos = newPos;
    _undoManager->pushRecord(record);
```

```
    // 4. 播放动画
    _gameView->playMultiMoveAnimation(cardIds, newPos, [](){});
}
```

5.3 修改视图层以支持多牌动画

在 GameView 中添加 playMultiMoveAnimation 方法，遍历卡牌 ID，为每个 CardView 播放移动动画。

```
void GameView::playMultiMoveAnimation(const std::vector<int>& cardIds, const std::vector<Vec2>& dests, const std::function<void()>& onComplete) {
    int count = cardIds.size();
    int finished = 0;
    for (int i = 0; i < count; ++i) {
        CardView* view = getCardViewById(cardIds[i]);
        if (view) {
            view->moveTo(dests[i], 0.3f, [&finished, count, onComplete]() {
                finished++;
                if (finished == count && onComplete) onComplete();
            });
        }
    }
}
```

5.4 在撤销管理器处理多牌回退

在 GameController::onUndoButtonClicked 中增加对新类型的处理：

```
void GameController::onUndoButtonClicked() {
    if (!_undoManager->canUndo()) return;
    UndoRecord record = _undoManager->popRecord();

    if (record.type == UndoType::TRAY_SWAP) {
        _trayController->undoSwap(record);
    } else if (record.type == UndoType::PLAYFIELD_TO_TRAY) {
        _playfieldController->undoPlayfieldToTray(record);
    } else if (record.type == UndoType::STOCK_TO_TRAY) {
        _stockController->undoStockToTray(record);
    } else if (record.type == UndoType::MULTI_MOVE) {
        // 反向移动：将卡牌从 newPos 移回 oldPos
        _gameView->playMultiMoveAnimation(record.cardIds,
record.oldPos, nullptr);
    }
}
```

总结：通过扩展 UndoRecord 结构、在控制器中生成新记录、在视图层添加动画支持、在撤销总控中添加分支，即可轻松支持多张牌同时回退。所有修改均在原有框架内进行，无需重构核心模块，体现了良好的扩展性。

6. 结论

本项目的 MVC 架构确保了各层职责清晰，核心扩展点（如添加一张功能卡牌“万能卡 CFT_JOKER”、多牌同时回退操作）均可通过局部修改实现，而无需侵入核心逻辑。本文所列举的扩展方法可推广至其他类似需求，如增加新的区域、新的匹配规则等。通过遵循本设计，我相信未来团队可以更高效、更安全地迭代新功能。